01 — Modelo de domínio (diagrama de classes)

Versão 1.4 · atualizado em 08/11/2026

Este documento descreve o modelo de domínio do **Continuum Control** e registra as decisões tomadas a partir do enunciado. As referências `§N` apontam para as seções do enunciado.

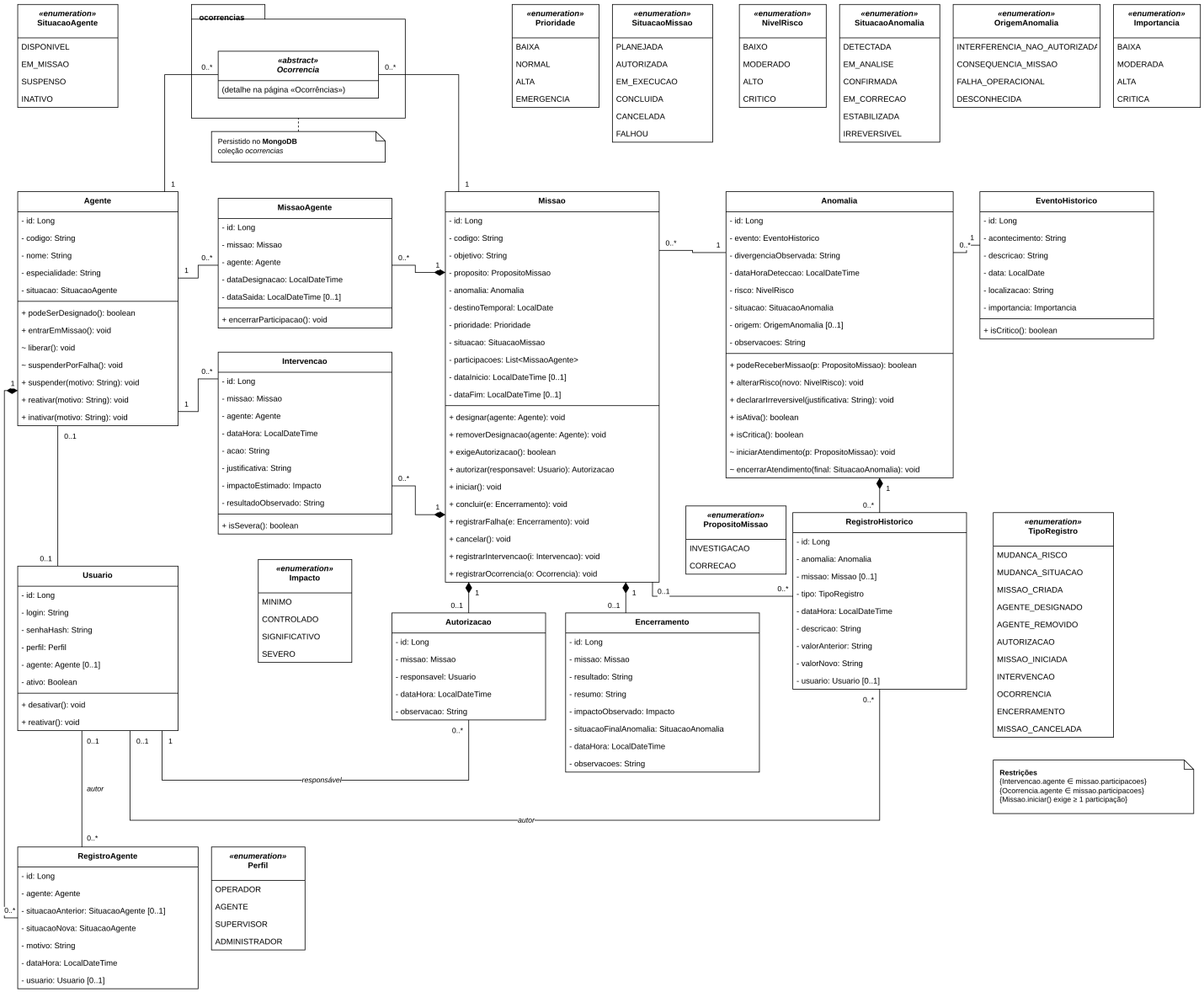
Fonte editável	`diagramas/continuum-control.drawio`, com as páginas *Diagrama principal* e *Ocorrências* (a página *DER* pertence ao doc 02, e as páginas *Estados — ** ao doc 03)
Rastreabilidade	`continuum-control-mapeamento.xlsx`: cada atributo, método e ligação com a seção de origem, marcado como explícito ou inferido
Documentos relacionados	00 — Enunciado · 02 — Modelo de dados · 03 — Estados · 04 — Persistência · 05 — API · 06 — Segurança

Sumário

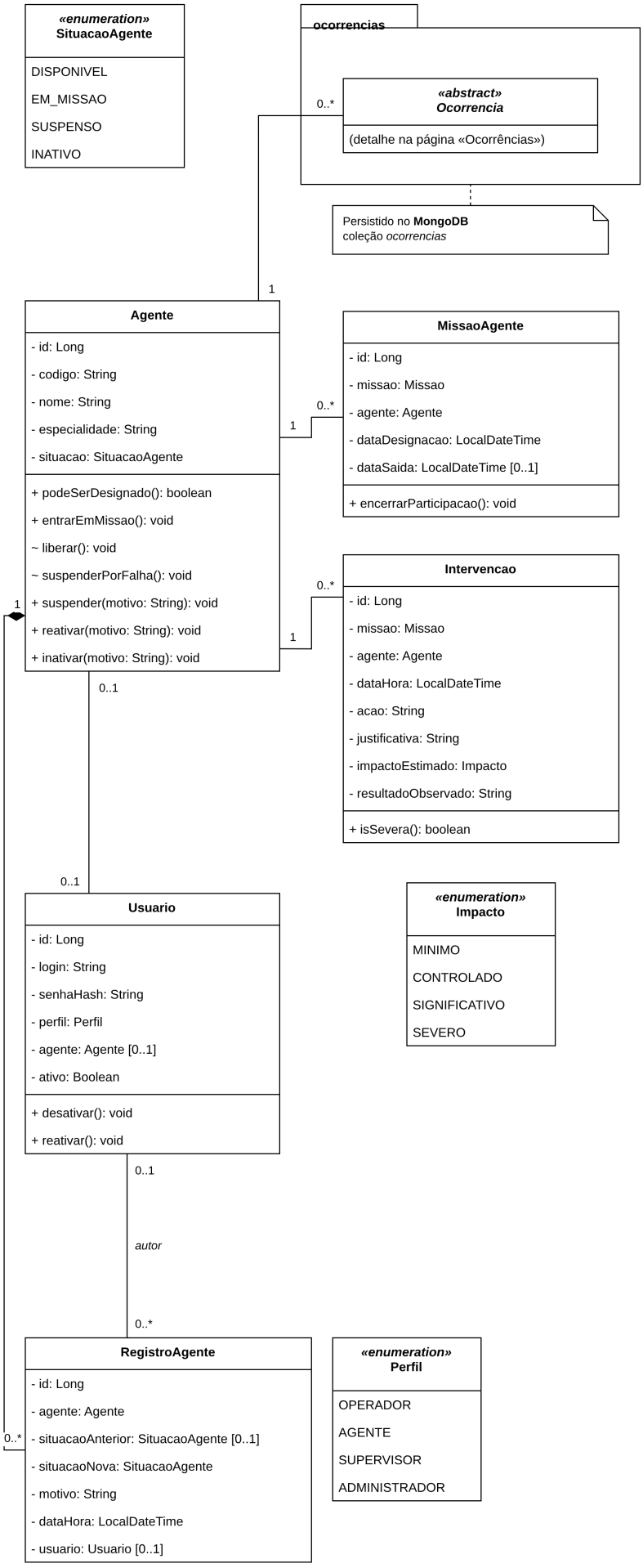
1. Diagramas
2. Visão geral
3. Convenções de notação
4. Enumerações
5. Relacionamentos
6. Regras de negócio e onde são garantidas
7. Decisões de modelagem
8. Restrições
9. Pontos em aberto
10. Histórico de revisões

1. Diagramas

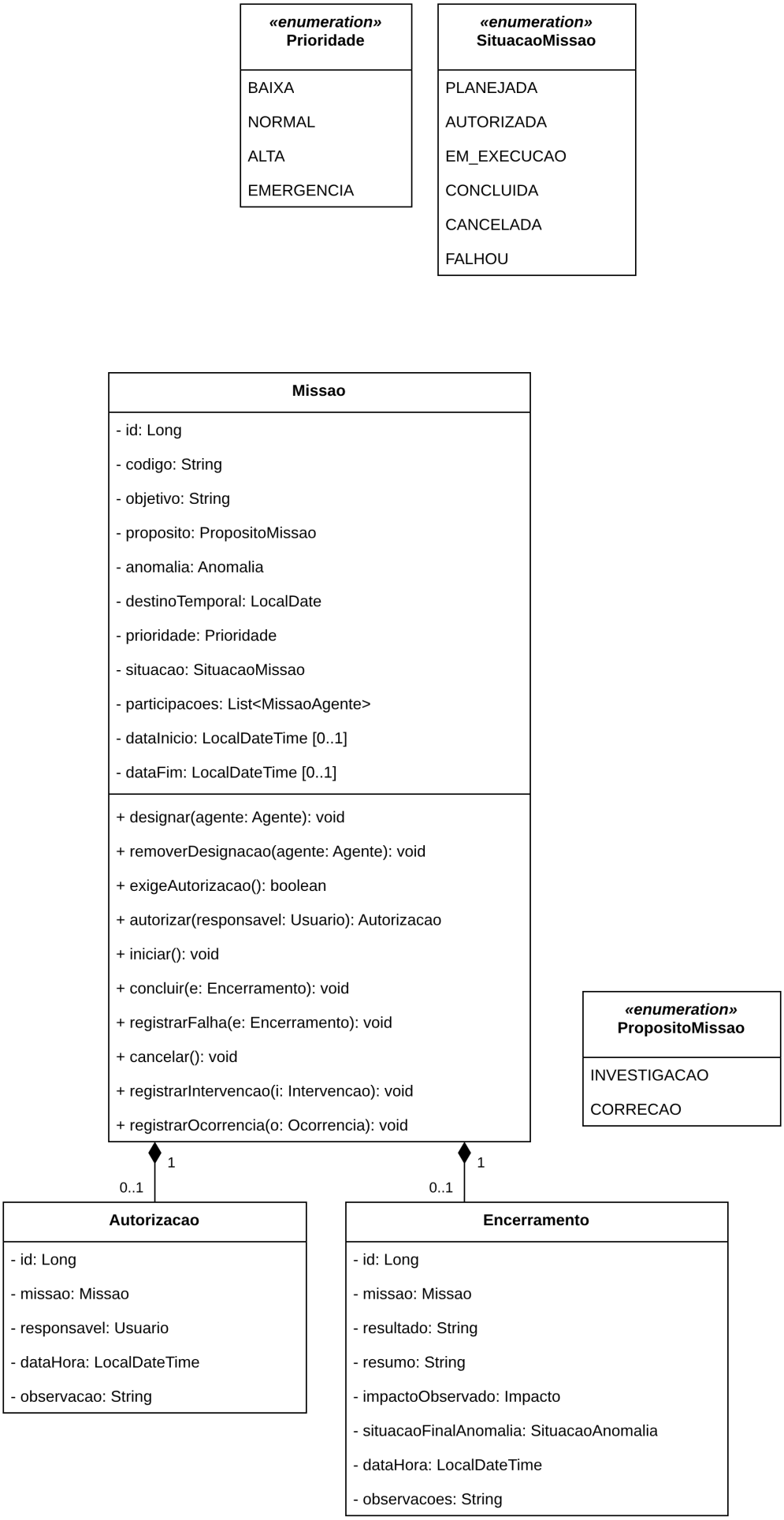
Diagrama principal



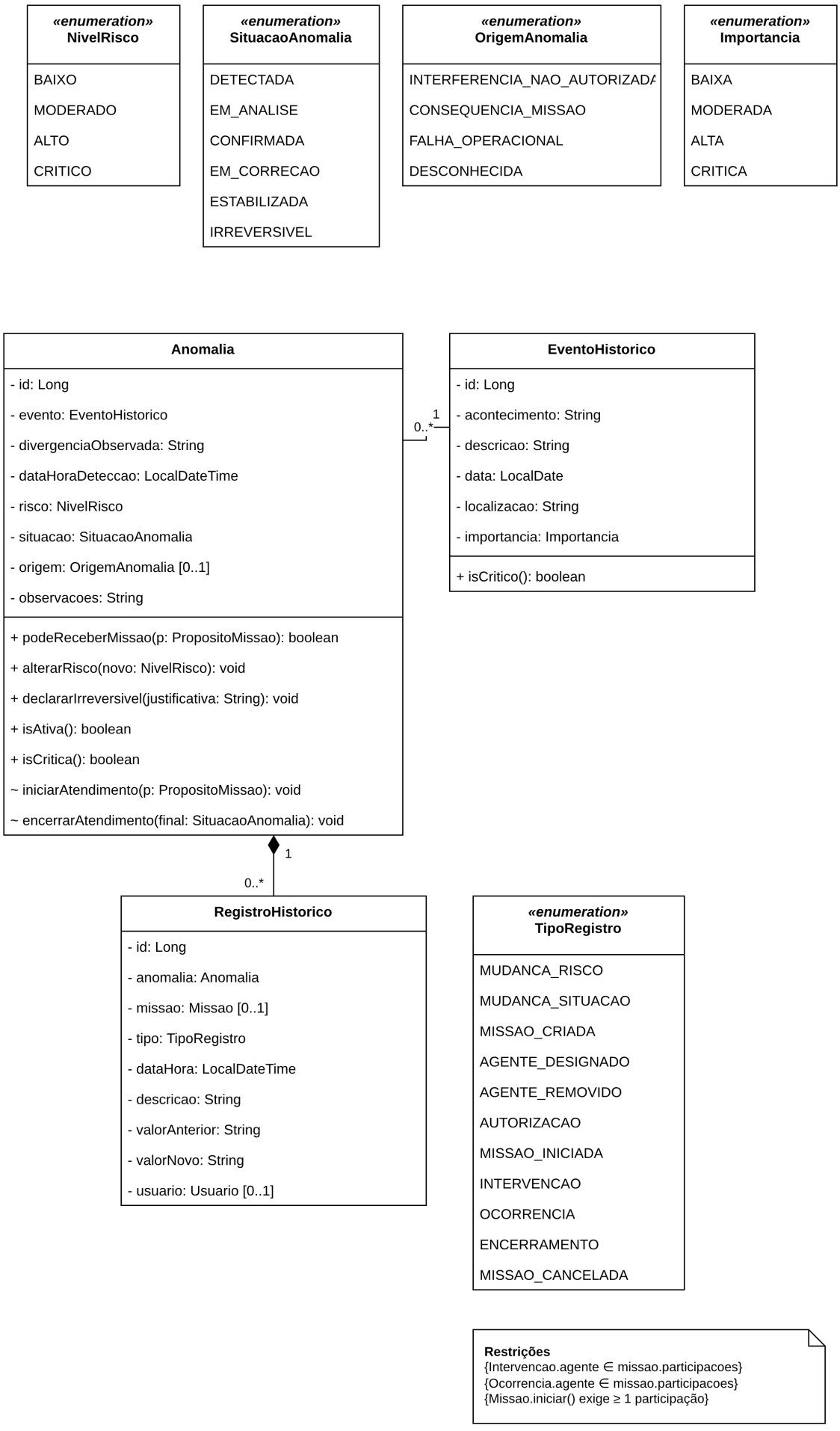
Visão geral · página *Diagrama principal* de `diagramas/continuum-control.drawio`. Ampliações legíveis nas três páginas seguintes.



Ampliação 1 de 3 · Agente, MissaoAgente, Intervencao, Usuario, RegistroAgente, pacote `ocorrencias` e enums ligados. As ligações com caixas de outras faixas aparecem na visão geral.



Ampliação 2 de 3 · Missao, Autorizacao, Encerramento e enums da missão. As ligações com caixas de outras faixas aparecem na visão geral.

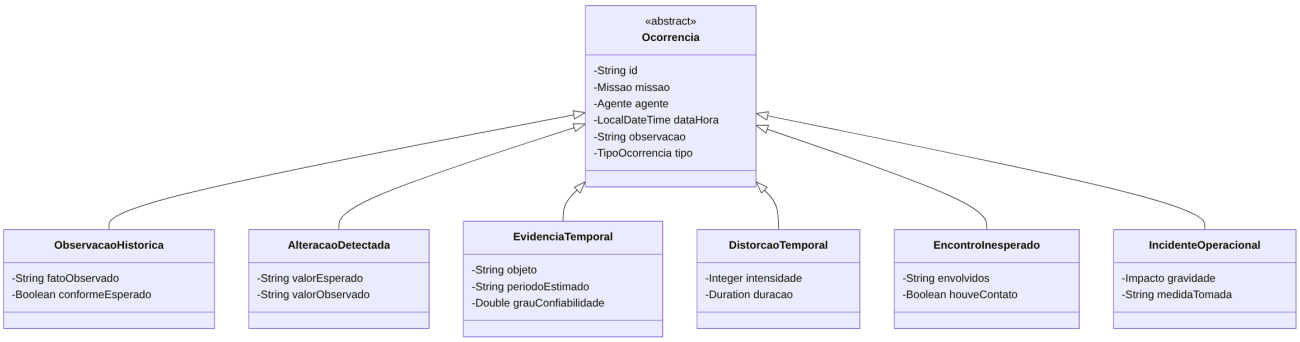


Ampliação 3 de 3 · Anomalia, EventoHistorico, RegistroHistorico e enums ligados. As ligações com caixas de outras faixas aparecem na visão geral.

> O GitHub não renderiza `.drawio`, por isso o diagrama é exportado em SVG (*File → Export as → SVG*, marcando a opção de exportar só a página atual). Sempre que o `.drawio` mudar, exporte de novo.

Pacote `ocorrencias`

No diagrama principal, o pacote aparece recolhido. O detalhe abaixo é renderizado pelo próprio GitHub:



Pacote `ocorrencias` · bloco Mermaid do documento

2. Visão geral

O modelo gira em torno de uma cadeia central:

```
Agente ── MissaoAgente ── Missao ── Anomalia ── EventoHistorico
```

1. Um **evento histórico** é um ponto monitorado da linha temporal (§4).
2. Uma **anomalia** é uma divergência detectada nesse evento (§5).
3. Uma **missão** investiga ou corrige a anomalia (§6).
4. **Agentes** participam da missão por meio de `MissaoAgente` (§7).

As demais classes registram o que acontece antes, durante e depois da missão:

Classe	Papel	§
`Autorizacao`	Liberação da missão por um supervisor	§8
`Intervencao`	Ação deliberada de um agente na linha temporal	§10
`Ocorrencia`	Informação observada durante a missão, com 6 subtipos	§11
`Encerramento`	Resultado final da missão	§13
`RegistroHistorico`	Linha do tempo de tudo o que aconteceu com uma anomalia	§12
`RegistroAgente`	Linha do tempo das mudanças de situação de um agente (D31)	§3, §12
`Usuario`	Quem acessa o sistema e com qual perfil	§22

3. Convenções de notação

Notação	Significado
`-` / `+` / `~`	Atributo privado / método público / método visível só no pacote de domínio
`nome: Tipo`	Tipos Java (ver tabela abaixo)
`[0..1]`	Atributo opcional (pode ser nulo)
Linha simples	Associação: as classes se conhecem, sem posse

Notação	Significado
◆ Diamante cheio	Composição: a parte não existe sem o todo; o diamante fica do lado do todo
△ Triângulo vazado	Herança: a ponta fica do lado da classe pai
`«abstract»` / *itálico*	Classe abstrata, que não é instanciada diretamente
`«enumeration»`	Conjunto fechado de valores
Pasta (*package*)	Agrupamento de classes

Tipo	Uso
`Long`	Identificador de entidades relacionais
`String` (id)	Identificador de documentos MongoDB (`ObjectId`)
`LocalDateTime`	Instante em que algo aconteceu no sistema (detecção, designação, autorização)
`LocalDate`	Data na linha temporal histórica (data do evento, destino da missão)
`Duration`	Intervalo de tempo (duração de uma distorção)

4. Enumerações

Enum	Valores	Usado em	§
`SituacaoAgente`	DISPONIVEL, EM_MISSAO, SUSPENSO, INATIVO	`Agente`	§3
`Importancia`	BAIXA, MODERADA, ALTA, CRITICA	`EventoHistorico`	§4
`NivelRisco`	BAIXO, MODERADO, ALTO, CRITICO	`Anomalia`	§5
`SituacaoAnomalia`	DETECTADA, EM_ANALISE, CONFIRMADA, EM_CORRECAO, ESTABILIZADA, IRREVERSIVEL	`Anomalia` , `Encerramento`	§5
`OrigemAnomalia` [1]	INTERFERENCIA_NAO_AUTORIZADA, CONSEQUENCIA_MISSAO, FALHA_OPERACIONAL, DESCONHECIDA	`Anomalia`	§1, §5
`Prioridade`	BAIXA, NORMAL, ALTA, EMERGENCIA	`Missao`	§6
`PropositoMissao` [1]	INVESTIGACAO, CORRECAO	`Missao`	§2, §5, §6
`SituacaoMissao`	PLANEJADA, AUTORIZADA, EM_EXECUCAO, CONCLUIDA, CANCELADA, FALHOU	`Missao`	§6
`Impacto`	MINIMO, CONTROLADO, SIGNIFICATIVO, SEVERO	`Intervencao` , `Encerramento` , `IncidenteOperacional`	§10
`TipoOcorrencia`	OBSERVACAO_HISTORICA, ALTERACAO_DETECTADA, EVIDENCIA_TEMPORAL, DISTORCAO_TEMPORAL, ENCONTRO_INESPERADO, INCIDENTE_OPERACIONAL	`Ocorrencia`	§11
`TipoRegistro` [1]	MUDANCA_RISCO, MUDANCA_SITUACAO, MISSAO_CRIADA, AGENTE_DESIGNADO, AGENTE_REMOVIDO, AUTORIZACAO, MISSAO_INICIADA, INTERVENCAO, OCORRENCIA, ENCERRAMENTO, MISSAO_CANCELADA	`RegistroHistorico`	§12
`Perfil`	OPERADOR, AGENTE, SUPERVISOR, ADMINISTRADOR	`Usuario`	§22
`EstadoLinhaTemporal` [1]	ESTAVEL, ATENCAO, INSTAVEL, CRITICO	Estado Operacional (calculado, D36)	§14

[1] Os valores não estão no enunciado; foram derivados do texto da seção indicada.

5. Relacionamentos

Ligação	Multiplicidade	Tipo	§
EventoHistorico — Anomalia	1 — 0..*	Associação	§5
Anomalia — Missao	1 — 0..*	Associação	§6, §13
Anomalia ◆ RegistroHistorico	1 — 0..*	Composição	§12
Missao ◆ MissaoAgente	1 — 0..*	Composição	§7
Missao ◆ Autorizacao	1 — 0..1	Composição	§8
Missao ◆ Intervencao	1 — 0..*	Composição	§10
Missao ◆ Encerramento	1 — 0..1	Composição	§13
Missao — Ocorrencia	1 — 0..*	Associação	§11
Agente — MissaoAgente	1 — 0..*	Associação	§7, §12
Agente — Intervencao	1 — 0..*	Associação	§10
Agente — Ocorrencia	1 — 0..*	Associação	§11
Ocorrencia △ 6 subclasses	—	Herança	§11
Usuario — Agente	0..1 — 0..1	Associação	§22
Usuario — Autorizacao (responsável)	1 — 0..*	Associação	§8, §22
Usuario — RegistroHistorico (autor)	0..1 — 0..*	Associação	§12, §22
Missao — RegistroHistorico	0..1 — 0..*	Associação	§12 (A10, D37 no doc 02)
Agente ◆ RegistroAgente	1 — 0..*	Composição	§3, §12 (D31)
Usuario — RegistroAgente (autor)	0..1 — 0..*	Associação	§22 (D31)

6. Regras de negócio e onde são garantidas

Cada regra fica na camada capaz de garanti-la sozinha:

Camada	Responsabilidade
Entrada (DTO)	Formato, obrigatoriedade e tamanho dos campos recebidos, com Bean Validation (`@NotBlank`, `@Size`...) → `400` (D32)
Entidade	Regras sobre o próprio estado. Ex.: uma missão `CONCLUIDA` não aceita intervenção. Parâmetros obrigatórios conferidos com `Objects.requireNonNull`
Serviço (caso de uso)	Orquestração entre entidades, a transação (§9) e a geração do histórico (§12)
Segurança	Perfil do usuário e posse do recurso (§22)
Banco	Integridade referencial, unicidade e não nulidade: última defesa, mesmo contra gravações fora da aplicação (ver doc 02)

6.1 Regras nas entidades

Método	Regra	Falha →	§
`Agente.podeSerDesignado()`	`true` apenas se `DISPONIVEL`	—	§3, §7
`Agente.entrarEmMissao()`	Passa de `DISPONIVEL` para `EM_MISSAO`	409	§7

Método	Regra	Falha →	§
`Agente.liberar()` ~	De `EM_MISSAO` para `DISPONIVEL` ao final da missão (G3); fora de `EM_MISSAO`, falha (D34)	409	§7
`Agente.suspenderPorFalha()` ~	De `EM_MISSAO` para `SUSPENSO` ao final de missão com falha (G4, D34)	409	§7
`Agente.suspender(motivo)`	Só a partir de `DISPONIVEL` (G5)	409	§3
`Agente.reativar(motivo)`	De `SUSPENSO` para `DISPONIVEL` (G6, D29)	409	§3
`Agente.inativar(motivo)`	A partir de `DISPONIVEL` ou `SUSPENSO`; substitui a remoção física e o agente permanece nos registros (G7)	409	§3, §12
`new Anomalia(evento, divergenciaObservada, risco, origem, observacoes, dataHoraDeteccao)`	Nasce `DETECTADA` (N1). `origem`, `observacoes` e `dataHoraDeteccao` são opcionais; `dataHoraDeteccao` vale *agora* quando omitida e não pode estar no futuro, pois a detecção pode anteceder o registro (§5)	400/409	§5
`Anomalia.podeReceberMissao(proposito)`	`true` se a anomalia está ativa, ou se está `IRREVERSIVEL` e a missão é de `INVESTIGACAO` (D14, doc 03 5.3)	409	§5
`Anomalia.declararIrreversivel(justificativa)`	De `DETECTADA` / `CONFIRMADA` para `IRREVERSIVEL` (N6). A justificativa é obrigatória e vai para o histórico; `observacoes` não muda (D33). "Sem missão em execução" é conferido no serviço	409	§5
`Anomalia.iniciarAtendimento(proposito)` ~	Chamado por `Missao.iniciar()`: vai para `EM_ANALISE` ou `EM_CORRECAO` (N2, N3)	409	§9
`Anomalia.encerrarAtendimento(situacaoFinal)` ~	Chamado no encerramento da missão (N4, N5). Confere só os destinos possíveis a partir da situação atual; a combinação propósito × desfecho é conferida na `Missao` (D35)	409	§13
`Missao.designar(agente)`	Só com a missão em `PLANEJADA` / `AUTORIZADA`, para agente disponível e ainda não designado	409	§7
`Missao.removerDesignacao(agente)`	Só em `PLANEJADA` / `AUTORIZADA` (M3, D25)	409	§7
`Missao.exigeAutorizacao()`	`true` se a prioridade é `EMERGENCIA` ou o risco da anomalia é `CRITICO` (A5)	—	§8
`Missao.autorizar(responsavel)`	Só a partir de `PLANEJADA`; cria a `Autorizacao` e passa a `AUTORIZADA`	409	§8
`Missao.iniciar()`	Exige ≥ 1 agente, autorização quando exigida, todos os agentes ainda `DISPONIVEL` e nenhuma outra missão em execução na anomalia (doc 03, 3.2)	409	§7, §8, §9
`Missao.registrarIntervencao()`	Só com a missão `EM_EXECUCAO`	409	§10
`Missao.concluir()` / `registrarFalha()`	Só a partir de `EM_EXECUCAO`; cria o `Encerramento` e confere a situação final contra propósito × desfecho (tabela 5.2 do doc 03, D35). Uma falha **não** estabiliza a anomalia	409	§13
`Missao.cancelar()`	Só a partir de `PLANEJADA` / `AUTORIZADA` (M7); também automático quando a anomalia deixa de admitir a missão (M8)	409	§6
`Intervencao.isSevera()`	`true` se o impacto é `SEVERO`; alimenta o índice	—	§15
`Usuario.desativar()` / `reativar()`	Alterna `ativo`; um usuário desativado não se autentica (doc 06)	409	§22

A coluna "Falha →" indica o status HTTP sugerido para a violação (§18). `409 Conflict` corresponde a "conflito com o estado atual do sistema".

6.2 Regras nos serviços

Caso de uso	O que é feito como uma única unidade (§9, §20)
Iniciar missão	Missão → `EM_EXECUCAO`, cada agente → `EM_MISSAO` e anomalia → `EM_ANALISE` ou `EM_CORRECAO`, conforme o `proposito` da missão (D14), com o registro no histórico. Se qualquer passo falhar, nada é gravado.

Caso de uso	O que é feito como uma única unidade (§9, §20)
Concluir ou falhar missão	Cria o `Encerramento`, atualiza missão e anomalia, libera os agentes e registra no histórico
Alterar risco ou situação da anomalia	Altera a anomalia e grava o `RegistroHistorico` correspondente
Mudar a situação de um agente	Toda transição de G1 a G7 grava um `RegistroAgente` na mesma transação, inclusive as automáticas de 6.1 e 6.2 do doc 03 (D31)
Registrar ocorrência	Confere a missão e o autor, grava o `RegistroHistorico` e a ocorrência no *outbox*, na mesma transação; a ocorrência chega ao MongoDB depois (doc 04, D40)

6.3 Regras de segurança

Perfis, regras de posse e a permissão de cada operação estão no doc 06. A permissão de cada endpoint está no doc 05.

---|---|---|

| Autorizar missão | SUPERVISOR, ADMINISTRADOR | — | | Registrar intervenção ou ocorrência | AGENTE | O agente do usuário precisa estar designado na missão | | Gerenciar agentes e usuários | ADMINISTRADOR | — |

> Esta tabela é preliminar. A matriz completa de permissões será definida junto com a API.

7. Decisões de modelagem

Cada decisão registra o **motivo** e, quando houve, a **alternativa descartada**.

D1 — `MissaoAgente` como classe de associação

A relação Missão × Agente é muitos-para-muitos e tem dados próprios (`dataDesignacao` e `dataSaida`). Isso permite reconstruir os "agentes envolvidos" de cada missão (§12) mesmo depois de liberados.

- *Descartada:* `@ManyToMany` simples, que não guarda as datas de entrada e saída.

D2 — Missão tem 0.. *agentes, e não 1..*

O §7 diz que a missão "poderá possuir um ou mais agentes", mas uma missão `PLANEJADA` é criada antes da designação. O mínimo de 1 agente vale **para iniciar**, então a regra fica em `Missao.iniciar()` e não na multiplicidade.

D3 — Composição sem exclusão em cascata

Autorização, Intervenção, Encerramento e `MissaoAgente` só existem no contexto de uma missão, por isso são composições. Como o §12 proíbe perder o histórico, **missões não são removidas**, e sim encerradas ou canceladas. A composição expressa dependência e posse, não `ON DELETE CASCADE`.

D4 — `Ocorrencia` abstrata com herança

O §11 afirma que as ocorrências não têm a mesma estrutura e dá exemplos de campos próprios para `DistorcaoTemporal` e `EvidenciaTemporal`.

- Os campos das outras quatro subclasses **não estão no enunciado**; foram propostos pelo grupo.
- `tipo` é redundante com a subclasse no modelo, mas serve de **discriminador** na persistência (ver doc 04).

- *Descartada*: uma classe única com um campo `detalhes: Map<String, Object>`, que perderia a tipagem e a validação dos campos.

D5 — Enums separados para importância e risco

`Importancia` (§4) e `NivelRisco` (§5) parecem iguais, mas classificam conceitos diferentes, de entidades diferentes, e podem evoluir separadamente. Os próprios valores diferem em gênero (BAIXA × BAIXO).

- *Descartada*: um enum único compartilhado.

D6 — `OrigemAnomalia` como enum opcional

O §5 pede "sua origem, quando conhecida", e o §1 lista as causas possíveis. Por isso o atributo é `[0..1]` e há o valor `DESCONHECIDA`, para quando a origem foi investigada sem conclusão.

- *Descartada*: texto livre, que inviabilizaria filtros e estatísticas por origem.

D7 — `Usuario` separado de `Agente`

Nem todo usuário é agente: operadores, supervisores e administradores também usam o sistema. A ligação `Usuario 0..1 — 0..1 Agente` permite a **autorização por recurso** do §22, em que um usuário `AGENTE` só altera missões nas quais o seu agente está designado.

- *Descartada*: `Agente` herdar de `Usuario`, o que obrigaria todo agente a ter login e misturaria identidade de acesso com dados operacionais.

D8 — `Encerramento` como classe própria

O §13 lista cinco informações do encerramento. Colocá-las em `Missao` deixaria vários campos nulos até o fim da missão. A classe separada marca de forma explícita que a missão foi encerrada.

D9 — `RegistroHistorico` só recebe inserções (append-only)

Cada mudança relevante gera um registro novo, que nunca é alterado. `valorAnterior` / `valorNovo` permitem reconstruir a sequência pedida no §12, e os valores de `TipoRegistro` seguem a lista do próprio §12.

D10 — Índice de Estabilidade e Estado Operacional fora do modelo

Os dois (§14, §15) são **calculados** a partir das entidades e não têm estado próprio. Na implementação, viram um serviço com DTOs de resposta, guardados em cache (doc 04). A fórmula está em D36.

D11 — Disponibilidade do agente conferida também ao iniciar

O agente continua `DISPONIVEL` enquanto a missão está apenas planejada, então pode ser designado para duas missões planejadas. A incompatibilidade do §7 é barrada em `iniciar()`, que confere de novo se todos os agentes ainda estão disponíveis.

- Duas missões iniciadas ao mesmo tempo com o mesmo agente são uma condição de corrida. Ela será tratada com *optimistic locking* (`@Version`) em `Agente` (§21).
- *Descartada*: marcar o agente como indisponível já na designação, o que bloquearia agentes em missões que podem nunca começar.

D12 — Atributos de referência e associações aparecem juntos

Atributos como `missao: Missao` repetem a informação da associação desenhada. As duas formas foram mantidas de propósito, para deixar explícita a navegabilidade e facilitar o mapeamento para o código (JPA).

D13 — Enums sem linha de dependência

O tipo no atributo (ex.: `situacao: SituacaoMissao`) já explicita a dependência. As linhas foram omitidas para não poluir o diagrama, e os enums ficam perto da classe que os usa.

D31 — `RegistroAgente` para as mudanças de situação do agente (fecha A12)

O histórico do §12 é organizado por anomalia, então suspensões, reativações e inativações de um agente não tinham onde ficar. Toda transição de situação do agente (G1 a G7, doc 03) grava um `RegistroAgente` com `situacaoAnterior`, `situacaoNova`, `motivo`, `dataHora` e `usuario` [0..1], sem nunca alterar os anteriores.

- `motivo` é obrigatório: informado pelo usuário nas transições manuais (G5–G7) e gerado pelo sistema nas automáticas (G2–G4, ex.: "Missão MI-7 iniciada"). No cadastro (G1), `situacaoAnterior` é nula.
- `usuario` é nulo nas transições automáticas.
- *Descartada*: colunas `motivo_situacao` em `agente`, que guardariam só o motivo atual e perderiam os anteriores.
- *Descartada*: reaproveitar `RegistroHistorico`, que pertence a uma anomalia (`anomalia` é obrigatória).

D32 — Validação em três camadas

- **DTO (Bean Validation):** formato, obrigatoriedade e tamanho do que chega pela API → `400`.
- **Entidade:** `Objects.requireNonNull` nos parâmetros obrigatórios e as guardas de estado → `409`.
- **Banco:** `NOT NULL`, `CHECK`, `FK` e `UNIQUE`, como última defesa.

As entidades **não** recebem `@NotNull`: o Hibernate só validaria na gravação, tarde demais para uma mensagem clara, e a regra ficaria duplicada.

D33 — A justificativa da irreversibilidade vai para o histórico

`declararIrreversivel(justificativa)` exige a justificativa, mas não a grava em `observacoes`. Ela vai para a `descricao` do `RegistroHistorico` do tipo `MUDANCA_SITUACAO`, junto com o autor e o instante.

- *Descartada:* substituir `observacoes`, que são as anotações da investigação (§5). Sobrescrevê-las apagaria informação, contra o §12.

D34 — Transições automáticas falham fora do estado esperado

`entrarEmMissao()`, `liberar()` e `suspenderPorFalha()` lançam a exceção de domínio (`409`) se o agente não estiver no estado de origem. Elas só são chamadas dentro das operações compostas do doc 03 (6.1 e 6.2); um estado inesperado ali indica erro de programação ou concorrência, e a falha desfaz a transação inteira (§9).

- `liberar()` e `suspenderPorFalha()` são visíveis só no pacote de domínio (`~`): nenhuma requisição chega a elas diretamente.
- `suspenderPorFalha()` é separada de `suspender(motivo)` porque a origem é outra (`EM_MISSAO` em vez de `DISPONIVEL`) e porque G4 não é uma ação do administrador.

- *Descartada*: ignorar a chamada em silêncio, o que esconderia o erro e deixaria a missão e o agente em estados incompatíveis.

D35 — A conferência da situação final fica dividida entre `Anomalia` e `Missao`

Cada classe confere o que consegue saber sozinha:

- `Anomalia.encerrarAtendimento(situacaoFinal)` aceita só os destinos possíveis a partir da situação atual: de `EM_ANALISE`, `DETECTADA`, `CONFIRMADA`, `ESTABILIZADA` ou `IRREVERSIVEL`; de `EM_CORRECAO`, `CONFIRMADA`, `ESTABILIZADA` ou `IRREVERSIVEL`; de `IRREVERSIVEL`, só `IRREVERSIVEL`.
- `Missao.concluir()` e `registrarFalha()` conferem a combinação propósito × desfecho da tabela 5.2 do doc 03.

A interseção das duas conferências é exatamente a tabela 5.2. Um teste deve percorrer todas as combinações e comparar com ela.

D36 — Fórmula do Índice de Estabilidade (§15) e estado geral (§14)

O §15 deixa a fórmula para o desenvolvimento. O índice parte de 100 e perde pontos por fator:

Fator (§15)	Penalidade padrão	O que é contado
Anomalias ativas, por risco	`BAIXO` 2 · `MODERADO` 5 · `ALTO` 10 · `CRITICO` 20	Anomalias ativas (doc 03, 5.3). As críticas pesam mais
Missões em execução	2 por missão	`situacao = EM_EXECUCAO`
Missões que falharam	8 por missão	`situacao = FALHOU` com `dataFim` nos últimos 30 dias
Intervenções severas	5 por intervenção	`impacto = SEVERO` com `dataHora` nos últimos 30 dias

`indice = max(0, 100 − soma das penalidades)`, como inteiro.

O estado geral da linha temporal (§14) deriva do índice:

Índice	Estado geral (`EstadoLinhaTemporal`)
80 a 100	`ESTAVEL`
50 a 79	`ATENCAO`
20 a 49	`INSTAVEL`
0 a 19	`CRITICO`

Exemplo: uma anomalia ativa de risco `ALTO`, uma `CRITICO`, uma missão em execução e uma intervenção severa no mês dão 100 − 10 − 20 − 2 − 5 = **63**, ou seja, `ATENCAO`.

- **Janela de 30 dias** para falhas e intervenções: fatos passados não podem rebaixar o índice para sempre. Anomalias ativas e missões em execução já são o estado atual.
- **Pesos e janela configuráveis** (`continuum.indice.*`, §24), sem recompilar.
- **Propriedades a testar:** o resultado fica sempre entre 0 e 100, e acrescentar um problema nunca aumenta o índice.
- O Estado Operacional (§14) reúne: agentes disponíveis e em missão, missões em execução, anomalias ativas e críticas, o índice e o estado geral. Os dois são calculados sob demanda e guardados em cache (doc 04).

8. Restrições

Estas restrições não cabem na notação das ligações:

Restrição	§	Garantida em
`Intervencao.agente ∈ missao.participacoes.agente`	§10	Banco (FK composta, D17 no doc 02)
`Ocorrencia.agente ∈ missao.participacoes.agente`	§11	Serviço (a ocorrência fica no MongoDB, sem FK)
Toda mudança de situação do agente gera um `RegistroAgente`	§3, §12	Serviço (D31)
Usuário com perfil `AGENTE` tem agente vinculado	§22	Banco (`ck_usuario_agente_perfil`)
`Missao.iniciar()` exige `participacoes.size() ≥ 1`	§7	Entidade
Toda missão nasce `PLANEJADA`	§6	Entidade (construtor)
`Anomalia.situacao = IRREVERSIVEL` ⇒ não aceita nova missão com `proposito = CORRECAO`	§5	Entidade
Um agente está em no máximo uma missão `EM_EXECUCAO`	§7	Entidade + `@Version` (D11)
Uma anomalia tem no máximo uma missão `EM_EXECUCAO`	§9	Banco (`uk_missao_em_execucao`, D23 no doc 03)
Um agente não é designado duas vezes para a mesma missão	§7	Entidade + `UNIQUE (missao_id, agente_id)`
`Missao.codigo` e `Agente.codigo` são únicos	§3, §6	Banco (`UNIQUE`)

O doc 02 (seção 7) detalha as restrições garantidas pelo banco.

9. Pontos em aberto

#	Questão	Proposta atual	Fechar em
A1	O que é uma anomalia **ativa** (§14, §16)?	Situação diferente de `ESTABILIZADA` e de `IRREVERSIVEL`	✅ Fechado no doc 03 (5.3)
A2	Quais transições valem para `suspender()` e `inativar()`?	Suspender a partir de `DISPONIVEL`; inativar a partir de `DISPONIVEL` ou `SUSPENSO`; `reativar()` de `SUSPENSO`	✅ Fechado no doc 03 (G5–G7)
A3	Que regras se aplicam a `cancelar()`? O enunciado só cita o estado `CANCELADA`	Permitido a partir de `PLANEJADA` e `AUTORIZADA`, manual ou automático	✅ Fechado no doc 03 (M7, M8)
A4	Qual é a exceção do §7 para o agente **não** voltar a `DISPONIVEL`?	Agente suspenso após uma falha em que **ele** registrou intervenção `SEVERO`	✅ Fechado no doc 03 (G4, D30)
A5	"Anomalias CRÍTICAS" no §8 se refere ao risco?	Sim: `risco = CRITICO`. É a única classificação da anomalia com o valor CRÍTICO (§2: "classificar o risco de anomalias"); a importância CRÍTICA é do evento (§4)	✅ Fechado
A6	`Encerramento.resultado` deve ser texto livre ou enum?	Texto livre; a situação final já está em `Missao.situacao`	✅ Fechado no doc 02
A7	`destinoTemporal` precisa de um local além da data?	Só `LocalDate`; o local já está no evento da anomalia	✅ Fechado no doc 02
A8	Que transição de anomalia acontece ao concluir a missão?	`Encerramento.situacaoFinalAnomalia`, restrita por propósito e desfecho	✅ Fechado no doc 03 (5.2)

#	Questão	Proposta atual	Fechar em
A9	O modelo não distingue missões de **investigação** e de **correção** (§2, §6), mas o §5 bloqueia só as de correção em anomalias irreversíveis	`proposito: PropositoMissao` (INVESTIGACAO, CORRECAO) em `Missao`	✅ Fechado (D14, doc 02)

10. Histórico de revisões

Versão	Data	Alteração
1.0	2026-10-21	Versão inicial
1.1	2026-10-21	Inclusão de enumerações, camadas de garantia das regras, alternativas descartadas, D11 (disponibilidade no início) e diagrama Mermaid do pacote `ocorrencias`
1.2	2026-10-21	`Missao.proposito` e enum `PropositoMissao` (fecha A9); A6 e A7 fechados no doc 02; restrição de intervenção passa a ser garantida pelo banco
1.3	2026-10-21	Ajustes vindos do doc 03: `Agente.reativar()`, `Missao.removerDesignacao()`, `Anomalia.alterarSituacao()` substituído por métodos de intenção, novos valores de `TipoRegistro`; A1–A4 e A8 fechados
1.4	2026-11-08	Ajustes vindos da implementação: `RegistroAgente` (D31, fecha A12), validação em três camadas (D32), justificativa no histórico (D33), `suspenderPorFalha()` e falha de `liberar()` fora de `EM_MISSAO` (D34), conferência da situação final (D35), fórmula do índice (D36), `Usuario.ativo`, `RegistroHistorico.missao`, construtor da `Anomalia`; A5 fechado